SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Pseudocode Design

Course Code:	**DSA0405**	**Course Title:**	**FUNDAMENTALS OF DATA SCIENCE**
CO Assessed:	**CO5 – Pseudocode Design**	**Assessment:**	**Assessment Tool 1 – Pseudocode Design**
Weightage:	**30% of CIA**	**Total Marks:**	**100**
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Kishore Kumar. P	**Reg. No.:**	192424108
Section / Batch:	2024	**Date:**	25/8/2026

Code of Conduct

I Kishore Kumar. P(192424108) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kshore Kumar. P

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

QUESTION 1: Supervised Learning

Problem Statement

Design a supervised learning workflow that accepts a labelled dataset, separates the features and target values, trains a model, and predicts the target for new data. The program should demonstrate the complete workflow using a simple numerical dataset and report a prediction and a basic evaluation measure.

Algorithm

1. Accept the labelled dataset containing input features and target values.

2. Separate the dataset into feature values (X) and target values (y).

3. Split the data into training and testing sets so that the model can be evaluated.

4. Train a supervised learning model using the training data.

5. Use the trained model to predict the target value for new input data.

6. Evaluate the model on test data and display the prediction and performance measure.

Code

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
X = np.array([[2, 60],[3, 65],[4, 70],[5, 75],[6, 80],[7, 85],[8, 90],[9, 95]])
y = np.array([45, 50, 55, 62, 68, 75, 82, 88])
 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
X_manual = np.c_[np.ones(len(X_train)), X_train]
theta = np.linalg.inv(X_manual.T @ X_manual) @ X_manual.T @ y_train
 def manual_predict(X_new):
    X_new = np.c_[np.ones(len(X_new)), X_new]
    return X_new @ theta

manual_prediction = manual_predict(np.array([[7, 85]]))

print("----- MANUAL IMPLEMENTATION -----")
```

```
print("Model coefficients:", theta)
print("Predicted marks for Hours=7, Attendance=85:",
      round(manual_prediction[0], 2))
model = LinearRegression()
model.fit(X_train, y_train)

prediction = model.predict([[7, 85]])

print("\n----- SCIKIT-LEARN IMPLEMENTATION -----")
print("Predicted marks:", round(prediction[0], 2))

test_prediction = model.predict(X_test)
error = mean_squared_error(y_test, test_prediction)
print("Mean Squared Error:", round(error, 2))
```

OUTPUT

```
----- MANUAL IMPLEMENTATION -----
Model coefficients: [-0.30163104  3.1418369   0.62763262]
Predicted marks for Hours=7, Attendance=85: 75.04

----- SCIKIT-LEARN IMPLEMENTATION -----
Predicted marks: 75.04
Mean Squared Error: 0.0
```

QUESTION 2: kNN Classifier

Problem Statement

Design pseudocode for a k-Nearest Neighbors (kNN) classifier that calculates the distance between a test sample and all training samples, selects the k nearest samples, and predicts the class using majority voting. The program should explicitly show the distances, nearest neighbours, and final prediction.

Algorithm

1. Accept the training samples and their corresponding class labels, and choose the value of k.

2. For the given test sample, calculate the Euclidean distance from the test sample to every training sample.

3. Sort all training samples in ascending order of their calculated distance.

4. Select the first k samples from the sorted list as the nearest neighbours.

5. Count the class labels among the k nearest neighbours and select the class with the highest vote.

6. Return the majority-vote class as the predicted class and verify it with the Scikit-learn implementation.

Code

```
import math
from collections import Counter
from sklearn.neighbors import KNeighborsClassifier
X_train = [[1, 1],[2, 2],[3, 2],[6, 6],[7, 7],[8, 6]]
y_train = ["Class A", "Class A", "Class A", "Class B", "Class B", "Class B"]
test_sample = [4, 4]
k = 3
def euclidean_distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
distances = []
for i in range(len(X_train)):
    distance = euclidean_distance(test_sample, X_train[i])
    distances.append((distance, y_train[i], X_train[i]))
```

```
distances.sort()
 print("----- MANUAL IMPLEMENTATION -----")
print("Distances:")
 for item in distances:
    print("Sample:", item[2],
          "Distance:", round(item[0], 2),
          "Class:", item[1])
nearest = distances[:k]
 print("\nNearest", k, "samples:")
for item in nearest:
    print(item[2], "->", item[1])
votes = Counter(item[1] for item in nearest)
prediction_manual = votes.most_common(1)[0][0]
print("\nManual Predicted Class:", prediction_manual)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
prediction = model.predict([test_sample])
print("\n----- SCIKIT-LEARN IMPLEMENTATION -----")
print("Predicted Class:", prediction[0])
```

OUTPUT

```
----- MANUAL IMPLEMENTATION -----
Distances:
Sample: [3, 2] Distance: 2.24 Class: Class A
Sample: [2, 2] Distance: 2.83 Class: Class A
Sample: [6, 6] Distance: 2.83 Class: Class B
Sample: [1, 1] Distance: 4.24 Class: Class A
Sample: [7, 7] Distance: 4.24 Class: Class B
Sample: [8, 6] Distance: 4.47 Class: Class B

Nearest 3 samples:
[3, 2] -> Class A
[2, 2] -> Class A
[6, 6] -> Class B

Manual Predicted Class: Class A

----- SCIKIT-LEARN IMPLEMENTATION -----
Predicted Class: Class A
```

QUESTION 3: kNN Classifier – Student Pass/Fail

Problem Statement

Design pseudocode to classify a new student as Pass or Fail using kNN based on attendance percentage, internal marks, and assignment scores. The program should identify the nearest students and use majority voting to determine the result.

Algorithm

1. Prepare the labelled student dataset using attendance percentage, internal marks, and assignment score as features, with Pass or Fail as the target class.

2. Choose a new student record and a suitable value of k.

3. Calculate the Euclidean distance between the new student and every student in the training dataset.

4. Sort the students by distance and select the k nearest students.

5. Use majority voting among the selected neighbours to determine Pass or Fail.

6. Compare the manual prediction with a Scikit-learn kNN model and calculate validation accuracy.

Code

```
import math
from collections import Counter
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X = [[95, 88, 92], [90, 85, 88], [85, 80, 82], [80, 75, 78], [75, 70, 72], [70, 65, 68], [65, 55, 60],
   [60, 50, 55], [55, 45, 50], [92, 90, 95]]
y = ["Pass", "Pass", "Pass", "Pass", "Pass", "Fail", "Fail", "Fail", "Fail", "Pass"]
new_student = [84, 78, 80]
k = 3
def distance(a, b):
   return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
```

```
distances = []

for i in range(len(X)):
    d = distance(new_student, X[i])
    distances.append((d, y[i], X[i]))
distances.sort()

print("----- MANUAL IMPLEMENTATION -----")
print("3 Nearest Students:")

nearest = distances[:k]

for d, result, student in nearest:
    print("Student:", student,
          "Distance:", round(d, 2),
          "Result:", result)
votes = Counter(result for _, result, _ in nearest)
manual_result = votes.most_common(1)[0][0]

print("\nManual Prediction:", manual_result)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

prediction = model.predict([new_student])

print("\n----- SCIKIT-LEARN IMPLEMENTATION -----")
print("New Student:")
print("Attendance:", new_student[0], "%")
print("Internal Marks:", new_student[1])
```

```
print("Assignment Score:", new_student[2])

print("Predicted Result:", prediction[0])
test_prediction = model.predict(X_test)
accuracy = accuracy_score(y_test, test_prediction)

print("Model Accuracy:", round(accuracy * 100, 2), "%")
```

OUTPUT

```
----- MANUAL IMPLEMENTATION -----
3 Nearest Students:
Student: [85, 80, 82] Distance: 3.0 Result: Pass
Student: [80, 75, 78] Distance: 5.39 Result: Pass
Student: [90, 85, 88] Distance: 12.21 Result: Pass

Manual Prediction: Pass

----- SCIKIT-LEARN IMPLEMENTATION -----
New Student:
Attendance: 84 %
Internal Marks: 78
Assignment Score: 80
Predicted Result: Pass
Model Accuracy: 100.0 %
```

QUESTION 4: kNN Classifier – Selecting the Appropriate k

Problem Statement

Design pseudocode for selecting the appropriate value of k in a kNN classifier by evaluating different k values on validation data. The program should test multiple k values, calculate validation accuracy for each value, and select the k that produces the best validation accuracy.

Algorithm

1. Prepare the labelled dataset and divide it into training data and validation data.

2. Choose a range of candidate k values to test, such as k = 1 to 7.

3. For each k value, predict the validation samples using kNN and calculate the validation accuracy.

4. Record the accuracy for every candidate k and compare the results.

5. Select the k value that gives the highest validation accuracy.

6. Confirm the selected k using the Scikit-learn kNN classifier and report the best validation accuracy.

Code

```
import math
from collections import Counter
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X = [
    [1, 1], [1, 2], [2, 1], [2, 2],
    [3, 3], [3, 4], [4, 3], [4, 4],
    [6, 6], [6, 7], [7, 6], [7, 7],
    [8, 8], [8, 9], [9, 8], [9, 9]]
y = [
    "A", "A", "A", "A",
    "A", "A", "A", "A",
    "B", "B", "B", "B",
    "B", "B", "B", "B"]
```

```
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42)
def distance(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))
 def predict_knn(train_X, train_y, sample, k):
    distances = []
     for i in range(len(train_X)):
        d = distance(sample, train_X[i])
        distances.append((d, train_y[i]))
     distances.sort()
    nearest = distances[:k]
    votes = Counter(label for _, label in nearest)
    return votes.most_common(1)[0][0]
 print("----- MANUAL IMPLEMENTATION -----")
best_k_manual = None
best_accuracy_manual = 0
 for k in range(1, 8):
    predictions = []
     for sample in X_val:
        prediction = predict_knn(
            X_train, y_train, sample, k)
        predictions.append(prediction)
     correct = sum(
        predictions[i] == y_val[i]
        for i in range(len(y_val)))
     accuracy = correct / len(y_val)
     print("k =", k, "Accuracy =", round(accuracy * 100, 2), "%")
     if accuracy > best_accuracy_manual:
        best_accuracy_manual = accuracy
        best_k_manual = k
print("\nBest k (Manual):", best_k_manual)
print("Best Validation Accuracy:",
```

```
      round(best_accuracy_manual * 100, 2), "%")
print("\n----- SCIKIT-LEARN IMPLEMENTATION -----")
 best_k = None
best_accuracy = 0
 for k in range(1, 8):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
     prediction = model.predict(X_val)
     accuracy = accuracy_score(y_val, prediction)
     print("k =", k,
          "Accuracy =", round(accuracy * 100, 2), "%")
     if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_k = k
 print("\nBest k (Scikit-learn):", best_k)
print("Best Validation Accuracy:",
      round(best_accuracy * 100, 2), "%")
```

OUTPUT

```
----- MANUAL IMPLEMENTATION -----
k = 1 Accuracy = 100.0 %
k = 2 Accuracy = 100.0 %
k = 3 Accuracy = 100.0 %
k = 4 Accuracy = 100.0 %
k = 5 Accuracy = 100.0 %
k = 6 Accuracy = 100.0 %
k = 7 Accuracy = 100.0 %

Best k (Manual): 1
Best Validation Accuracy: 100.0 %

----- SCIKIT-LEARN IMPLEMENTATION -----
k = 1 Accuracy = 100.0 %
k = 2 Accuracy = 100.0 %
k = 3 Accuracy = 100.0 %
k = 4 Accuracy = 100.0 %
k = 5 Accuracy = 100.0 %
k = 6 Accuracy = 100.0 %
k = 7 Accuracy = 100.0 %

Best k (Scikit-learn): 1
Best Validation Accuracy: 100.0 %
```

QUESTION 5: Decision Tree

Problem Statement

Design pseudocode to construct a Decision Tree classifier by selecting the best feature for splitting the training dataset and recursively creating child nodes. The program should calculate entropy and information gain for candidate splits, build a simple recursive tree, predict a new sample, and show an equivalent Scikit-learn tree.

Algorithm

1. Prepare the training dataset with features and class labels, and define the entropy function for measuring impurity.

2. For each feature, consider candidate threshold values and divide the training data into left and right child groups.

3. Calculate the information gain for each possible split and select the feature and threshold with the highest gain.

4. Create child nodes from the selected split and recursively repeat the same process for each child until a stopping condition is reached.

5. Use the completed tree to traverse from the root to a leaf and predict the class for a new sample.

6. Build an equivalent Scikit-learn Decision Tree using entropy as the splitting criterion and compare the prediction.

Code

```
import math
from collections import Counter
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
X = [[90, 85], [85, 80], [80, 75], [75, 70], [70, 65], [65, 60], [60, 55], [55, 50], [95, 90], [88, 82]]
 y = ["Pass", "Pass", "Pass", "Pass", "Pass", "Fail", "Fail", "Fail", "Pass", "Pass"]
 feature_names = ["Attendance", "Internal Marks"]
def entropy(labels):
    total = len(labels)
    counts = Counter(labels)
```

```
    result = 0
    for count in counts.values():
        probability = count / total
        result -= probability * math.log2(probability)
    return result
def split_dataset(X, y, feature, threshold):
    left_X = []
    left_y = []
    right_X = []
    right_y = []
    for i in range(len(X)):
        if X[i][feature] <= threshold:
            left_X.append(X[i])
            left_y.append(y[i])
        else:
            right_X.append(X[i])
            right_y.append(y[i])
    return left_X, left_y, right_X, right_y
 def information_gain(X, y, feature, threshold):
    parent_entropy = entropy(y)
    left_X, left_y, right_X, right_y = split_dataset(
        X, y, feature, threshold)
    if len(left_y) == 0 or len(right_y) == 0:
        return 0
    left_weight = len(left_y) / len(y)
    right_weight = len(right_y) / len(y)
    child_entropy = (
        left_weight * entropy(left_y)
        + right_weight * entropy(right_y))
    return parent_entropy - child_entropy
best_gain = -1
best_feature = None
best_threshold = None
```

```
 for feature in range(len(X[0])):
    values = sorted(set(row[feature] for row in X))
    for i in range(len(values) - 1):
        threshold = (values[i] + values[i + 1]) / 2
        gain = information_gain(
           X, y, feature, threshold)
       if gain > best_gain:
           best_gain = gain
           best_feature = feature
           best_threshold = threshold
print("----- MANUAL IMPLEMENTATION -----")
print("Best Feature:", feature_names[best_feature])
print("Best Threshold:", best_threshold)
print("Information Gain:", round(best_gain, 4))
class Node:
   def __init__(
       self,
       feature=None,
       threshold=None,
       left=None,
       right=None,
       prediction=None
   ):
       self.feature = feature
       self.threshold = threshold
       self.left = left
       self.right = right
       self.prediction = prediction
def build_tree(X, y, depth=0, max_depth=3):
    if len(set(y)) == 1:
        return Node(prediction=y[0])
    if depth >= max_depth:
        return Node(
```

```
            prediction=Counter(y).most_common(1)[0][0])
    best_gain = -1
    best_feature = None
    best_threshold = None
    for feature in range(len(X[0])):
        values = sorted(set(row[feature] for row in X))
        for i in range(len(values) - 1):
            threshold = (values[i] + values[i + 1]) / 2
            gain = information_gain(
                X, y, feature, threshold)
            if gain > best_gain:
                best_gain = gain
                best_feature = feature
                best_threshold = threshold
    if best_gain <= 0:
        return Node(
            prediction=Counter(y).most_common(1)[0][0])
    left_X, left_y, right_X, right_y = split_dataset(
        X, y, best_feature, best_threshold)
    left_child = build_tree(
        left_X, left_y, depth + 1, max_depth)
    right_child = build_tree(
        right_X, right_y, depth + 1, max_depth
    )
    return Node(
        feature=best_feature,
        threshold=best_threshold,
        left=left_child,
        right=right_child
    )
def predict_tree(node, sample):
    if node.prediction is not None:
        return node.prediction
```

```
    if sample[node.feature] <= node.threshold:
        return predict_tree(node.left, sample)
    else:
        return predict_tree(node.right, sample)
tree = build_tree(X, y)
new_student = [82, 78]
manual_prediction = predict_tree(
    tree, new_student)
print("\nNew Student:", new_student)
print("Manual Prediction:", manual_prediction)
model = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=3,
    random_state=42)
model.fit(X, y)
prediction = model.predict([new_student])

print("\n----- SCIKIT-LEARN IMPLEMENTATION -----")
print("New Student:", new_student)
print("Predicted Result:", prediction[0])

plt.figure(figsize=(10, 6))

plot_tree(
    model,
    feature_names=feature_names,
    class_names=model.classes_,
    filled=True
)

plt.title("Decision Tree")
plt.show()
```

OUTPUT

```
----- MANUAL IMPLEMENTATION -----
Best Feature: Attendance
Best Threshold: 67.5
Information Gain: 0.8813

New Student: [82, 78]
Manual Prediction: Pass

----- SCIKIT-LEARN IMPLEMENTATION -----
New Student: [82, 78]
Predicted Result: Pass
```